

[Event dashboard](#) > Building the Command-Line Agent

Building the Command-Line Agent

Now we will build the agent step by step. Each function we add corresponds to one of the four building blocks introduced in section 1:

Function	Building Block
<code>call_claude_sonnet()</code>	The Brain
<code>execute_curl_command()</code>	The Hands
<code>generate_weather_api_calls()</code>	The Planner
<code>process_weather_response()</code>	The Analyst

Step 1: Amazon Bedrock Connection — The Brain (8 minutes)

We will start with the foundation: a single function that sends a prompt to Claude 4.5 Sonnet and returns the response. Every other building block will call into this one.

Inside the folder created in the environment setup i.e. `agentic-ai-workshop`, create a new file called `weather_agent_cli.py`  and add this code:

```
1  # Import necessary libraries
2  import boto3          # AWS SDK for Python - allows us to interact with AWS servi
3  import json           # For handling JSON data
4  import subprocess     # For running system commands like curl
5  import time           # For adding delays and timing operations
6  from datetime import datetime # For timestamps and date operations
7
8  def call_claude_sonnet(prompt):
9      """
10     This function sends a prompt to Claude 4.5 Sonnet and gets a response.
11     This is the "brain" of our agent - where the AI thinking happens.
12
```

```

13     Args:
14         prompt (str): The question or instruction we want to send to Claude
15
16     Returns:
17         tuple: (success: bool, response: str) - success status and Claude's res
18     """
19     # Create a connection to Amazon Bedrock service
20     # Bedrock is AWS's service for accessing AI models like Claude
21     bedrock = boto3.client(
22         service_name='bedrock-runtime', # Specify we want the runtime version
23         region_name='us-west-2'        # AWS region - using us-west-2 as spec
24     )
25
26     try:
27         # Send our prompt to Claude and get a response
28         response = bedrock.converse(
29             # Specify which version of Claude we want to use
30             modelId='us.anthropic.claude-sonnet-4-5-20250929-v1:0', # Claude 4
31
32             # Format our message - Claude expects messages in a specific struct
33             messages=[
34                 {
35                     "role": "user", # We are the user asking
36                     "content": [{"text": prompt}] # Our actual question/pr
37                 }
38             ],
39
40             # Configure how Claude should respond
41             inferenceConfig={
42                 "maxTokens": 2000, # Maximum length of response (tokens ≈ wo
43                 "temperature": 0.7 # Creativity level (0=very focused, 1=very
44             }
45         )
46
47         # Extract the actual text response from Claude's response structure
48         # The response comes nested in a complex structure, so we dig down to g
49         return True, response['output']['message']['content'][0]['text']
50
51     except Exception as e:
52         # If something goes wrong, return an error message
53         return False, f"Error calling Claude: {str(e)}"
54
55     # Test our connection to Claude
56     if __name__ == "__main__":
57         print("🔧 Testing connection to Claude 4.5 Sonnet...")
58         success, response = call_claude_sonnet("Hello! Are you working today?")
59
60         if success:
61             print("✅ Connection successful!")

```

```
62     print(f"Claude says: {response}")
63 else:
64     print("❌ Connection failed!")
65     print(f"Error: {response}")
```

Test this code:

Now, in the terminal window, run this:

```
1 python weather_agent_cli.py
```



You should see Claude's response confirming the connection works.

What's Happening Here

1. **Bedrock client** — we create a connection to Amazon Bedrock in `us-west-2`.
2. **Model selection** — we ask for Claude 4.5 Sonnet by ID.
3. **Message format** — Claude expects messages as a list of role/content pairs.
4. **Inference config** — `maxTokens` caps the reply length; `temperature` controls how deterministic the output is.
5. **Error handling** — any `boto3` exception is captured and returned, rather than crashing the agent.

Step 2: Building Agent Logic — Hands, Planner, Analyst (10 minutes)

IMPORTANT: Make sure to REMOVE the code underneath **# Test our connection to Claude** from the previous step before pasting the code below!

Next we add the other three building blocks on top of the Brain:

- **The Hands** — `execute_curl_command()` makes HTTP requests to the real world
- **The Planner** — `generate_weather_api_calls()` asks Claude to produce the right API URL
- **The Analyst** — `process_weather_response()` asks Claude to summarise the JSON

Add these functions to your `weather_agent_cli.py` file:



```

1  def execute_curl_command(url):
2      """
3      Execute a curl command to fetch data from an API.
4      This is how our agent "acts" in the real world - making HTTP requests.
5
6      Args:
7          url (str): The URL to fetch data from
8
9      Returns:
10         tuple: (success: bool, response: str) - success status and API respons
11     """
12     try:
13         # Use curl command to make HTTP request
14         # curl is a command-line tool for making HTTP requests
15         result = subprocess.run(
16             ['curl', '-s', url], # -s flag makes curl silent (no progress inf
17             capture_output=True, # Capture the output so we can process it
18             text=True,          # Return output as text (not bytes)
19             timeout=30          # Give up after 30 seconds
20         )
21
22         # Check if the command was successful
23         if result.returncode == 0:
24             return True, result.stdout
25         else:
26             return False, f"Curl command failed: {result.stderr}"
27
28     except subprocess.TimeoutExpired:
29         return False, "Request timed out after 30 seconds"
30     except Exception as e:
31         return False, f"Error executing curl: {str(e)}"
32
33 def generate_weather_api_calls(location):
34     """
35     Use Claude to intelligently generate National Weather Service API calls fo
36     This is where the "agentic" magic happens - AI planning the API calls.
37
38     Args:
39         location (str): The location provided by the user
40
41     Returns:
42         tuple: (success: bool, api_calls: list) - success status and list of A
43     """
44     # Create a detailed prompt that teaches Claude how to generate NWS API cal
45     prompt = f"""

```

```

46 You are an expert at working with the National Weather Service (NWS) API.
47
48 Your task: Generate the NWS API URL to get weather forecast data for "{location}"
49
50 Instructions:
51 1. First, determine the approximate latitude and longitude coordinates for this location.
52 2. Generate the NWS Points API URL: https://api.weather.gov/points/{lat},{lon}
53
54 For the coordinates, use your knowledge to estimate:
55 - Major cities: Use well-known coordinates
56 - ZIP codes: Estimate based on the area
57 - States: Use approximate center coordinates
58 - In case a location description is provided instead of a location name, please use the location description.
59
60 Example for Seattle:
61 https://api.weather.gov/points/47.6062,-122.3321
62
63 Example for largest city in USA:
64 Based on your knowledge, you will establish location is New York City
65 https://api.weather.gov/points/40.7128,-74.0060
66
67 Now generate the API call (Points API) for the established location.
68 Return ONLY the complete Points API URL, nothing else.
69 Format: https://api.weather.gov/points/LAT,LON
70 """
71
72 print(f"🧠 AI is analyzing '{location}' and generating weather API calls.
73 success, response = call_claude_sonnet(prompt)
74
75 if success:
76     # Clean up the response - sometimes Claude adds extra text
77     api_url = response.strip()
78     # Make sure we got a valid URL
79     if api_url.startswith('https://api.weather.gov/points/'):
80         return True, [api_url] # Return as list for consistency
81     else:
82         return False, f"AI generated invalid URL: {api_url}"
83 else:
84     return False, response
85
86 def get_forecast_url_from_points_response(points_json):
87     """
88     Extract the forecast URL from the NWS Points API response.
89
90     Args:
91         points_json (str): JSON response from the Points API
92
93     Returns:
94         tuple: (success: bool, forecast_url: str) - success status and forecast URL

```

```

95     """
96     try:
97         data = json.loads(points_json)
98         forecast_url = data['properties']['forecast']
99         return True, forecast_url
100    except (json.JSONDecodeError, KeyError) as e:
101        return False, f"Error parsing Points API response: {str(e)}"
102
103    def process_weather_response(raw_json, location):
104        """
105        Use Claude to convert raw NWS API JSON into a human-readable weather summary
106        This is where AI processes complex data into useful information.
107
108        Args:
109            raw_json (str): Raw JSON response from NWS API
110            location (str): Original location for context
111
112        Returns:
113            tuple: (success: bool, summary: str) - success status and processed su
114        """
115        prompt = f"""
116        You are a weather information specialist. I have raw National Weather Service
117
118        Raw NWS API Response:
119        {raw_json}
120
121        Please create a weather summary that includes:
122        1. A brief introduction with the location
123        2. Current conditions and today's forecast
124        3. The next 2-3 days outlook with key details (temperature, precipitation, win
125        4. Any notable weather patterns or alerts
126        5. Format the response to be easy to read and understand
127
128        Make it informative and practical for someone planning their activities. Focus
129        """
130
131        print(f"🌤️ AI is processing weather data and creating summary...")
132        success, response = call_claude_sonnet(prompt)
133
134        return success, response

```

What's Happening Here

1. `execute_curl_command()` — **The Hands:** runs the actual HTTP request via `subprocess`.

2. `generate_weather_api_calls()` — **The Planner:** asks Claude to produce a valid Points API URL.
3. `get_forecast_url_from_points_response()` — a deterministic JSON parse. No model call: the URL is right there in `properties.forecast`.
4. `process_weather_response()` — **The Analyst:** feeds the raw forecast JSON back to Claude and asks for a readable summary.

Step 3: Main Agent Workflow (5 minutes)

Now we tie the four functions together with a main loop that runs the six-step workflow from the architecture page: AI Planning → Points API → Extract URL → Forecast API → AI Analysis → Display. Add this to your file:



```

1  def run_weather_agent():
2      """
3      Main function that orchestrates our AI agent.
4      This demonstrates the complete agentic workflow.
5      """
6      print("🌤️ Welcome to the Weather AI Agent!")
7      print("This agent uses Claude 4.5 Sonnet to help you get weather forecasts.")
8      print("=" * 60)
9
10     while True:
11         # Get user input
12         location = input("\n🔍 Enter a location name or description (or 'quit'
13
14         if location.lower() in ['quit', 'exit', 'q']:
15             print("👋 Thanks for using the Weather Agent!")
16             break
17
18         if not location:
19             print("❌ Please enter a location name or description.")
20             continue
21
22         print(f"\n🚀 Starting weather analysis for '{location}'...")
23         print("-" * 40)
24
25         # Step 1: AI generates the Points API URL
26         print("Step 1: 🗨️ AI Planning Phase")
27         success, api_calls = generate_weather_api_calls(location)
28
29         if not success:
30             print(f"❌ Failed to generate API calls: {api_calls}")

```

```

31         continue
32
33     points_url = api_calls[0]
34     print(f"✅ Generated Points API URL: {points_url}")
35
36     # Step 2: Execute the Points API call
37     print("\nStep 2: 🔗 Points API Execution")
38     print("Fetching location data from National Weather Service...")
39     success, points_response = execute_curl_command(points_url)
40
41     if not success:
42         print(f"❌ Failed to fetch points data: {points_response}")
43         continue
44
45     print(f"✅ Received points data")
46
47     # Step 3: Extract forecast URL from Points response
48     print("\nStep 3: 📌 Extracting Forecast URL")
49     success, forecast_url = get_forecast_url_from_points_response(points_re
50
51     if not success:
52         print(f"❌ Failed to extract forecast URL: {forecast_url}")
53         continue
54
55     print(f"✅ Forecast URL: {forecast_url[:60]}...")
56
57     # Step 4: Execute the Forecast API call
58     print("\nStep 4: 🌤️ Forecast API Execution")
59     print("Fetching weather forecast data...")
60     success, forecast_response = execute_curl_command(forecast_url)
61
62     if not success:
63         print(f"❌ Failed to fetch forecast data: {forecast_response}")
64         continue
65
66     print(f"✅ Received {len(forecast_response)} characters of forecast da
67
68     # Step 5: AI processes the response
69     print("\nStep 5: 🧠 AI Analysis Phase")
70     success, summary = process_weather_response(forecast_response, location
71
72     if not success:
73         print(f"❌ Failed to process data: {summary}")
74         continue
75
76     # Step 6: Display results
77     print("\nStep 6: 🗨️ Weather Forecast")
78     print("=" * 60)
79     print(summary)

```

```
80         print("=" * 60)
81
82         print(f"\n✅ Weather analysis complete for '{location}'!")
83
84     # Run the agent when the script is executed
85     if __name__ == "__main__":
86         run_weather_agent()
```

Complete Agent Workflow

The agent now implements the full Problem → Plan → Action → Result loop in six steps:

Step	Phase	Building block
1. AI Planning	Plan	Planner calls Brain
2. Points API	Action	Hands
3. Extract URL	Plan	Deterministic parse
4. Forecast API	Action	Hands
5. AI Analysis	Result	Analyst calls Brain
6. Display	Result	Terminal output

Test Your Agent

In the terminal, run your completed agent:

```
1 python weather_agent_cli.py
```



Try these inputs to see how the planner handles different formats:

- Seattle — major city, well-known coordinates
- 90210 — ZIP code resolution
- New York City — multi-word city name

- National park near Homestead in Florida — descriptive query that should resolve to the Everglades
- Largest City in California — knowledge-based resolution to Los Angeles

You now have a working agent. The same logic powers the web app in the next section.

- **If you had any issues with the step-by-step code snippets above, here's the complete ``weather_agent_cli.py`` file for reference:**

Next, let's see this same functionality in a beautiful web interface!

[Previous](#)[Next](#)